

CYBR 437: Secure Coding

Secure Coding in C-Part II

Instructor: Abinash Borah

Outline

- Secure Memory Management in C
- Mitigation Strategies for Heap-based Vulnerabilities
- Activity (?)

Memory Management in C

- Memory management has been a source of programming defects, security flaws, and vulnerabilities
- Dynamic memory management in C programs can be complicated and consequently prone to various errors
- Common programming errors related to memory management include:
 - Initialization errors
 - Failing to check return values
 - Dereferencing null or invalid pointers
 - Referencing freed memory
 - Freeing the same memory multiple times
 - Memory leaks

Lifetime of Allocated Memory

- Memory allocated by the C memory allocation functions has an allocated storage duration
- Storage duration is the property of an object that defines the minimum potential lifetime of the storage containing the object
- The lifetime of these objects is not restricted to the scope in which they are created
- For example, if `malloc()` is called within a function, the allocated memory will still exist after the function returns

Initialization Errors

- The malloc() function is commonly used to allocate blocks of memory
- The value at the space returned by malloc() is indeterminate
- A common error is incorrectly assuming that malloc() initializes this memory to all bits zero
- Points to remember:
 - Do not assume that memory allocation functions initialize memory
 - Do not reference uninitialized memory

```
/* return y = Ax */
int *matvec(int **A, int *x, int n) {
    int *y = malloc(n * sizeof(int));
    int i, j;

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            y[i] += A[i][j] * x[j];
    return y;
}
```

Failing to Check Return Values

- C memory allocation functions return a null pointer if the requested memory allocation fails – programmers need to determine if an allocation fails and manage it properly
- Failure to detect and properly handle memory allocation failures can lead to unpredictable and unintended program behavior
- For example, older versions of Adobe Flash neglected to check the return value from `calloc()`, which resulted in a vulnerability
- Even when `calloc()` returns a null pointer, Flash writes to an offset from the return value
- Dereferencing a null pointer usually results in a program crash, but dereferencing an offset from a null pointer allows an exploit to succeed without crashing the program

A vulnerability example

- Assume that temp_num, tmp2, and num_of_records are under the control of an attacker in this example
- The attacker can cause malloc() to fail by providing a large value for num_of_records; when malloc() fails, it returns a null pointer that is assigned to start
- The value of temp_num is scaled by the size of signal_info when added to start, and the resulting pointer value is stored in point
- To exploit this vulnerability, the attacker can supply a value for temp_num that results in point referencing a writable address to which control is eventually transferred
- The memory at that address is overwritten by the contents of the string referenced by tmp2, resulting in an arbitrary code execution vulnerability
- This vulnerability could be eliminated by simply testing that the pointer returned by malloc() is not null

```
signal_info *start = malloc(num_of_records * sizeof(signal_info));  
signal_info *point = (signal_info *)start;  
point = start + temp_num - 1;  
memcpy (point->sig_desc, tmp2, strlen(tmp2));
```

Dereferencing Null or Invalid Pointers

- Dereferencing a null pointer typically results in a segmentation fault, but this is not always the case
- For example, many Cray supercomputers had memory mapped at address 0, so it worked just like any other memory reference
- Many embedded systems work the same way
- Other embedded systems have registers mapped at address 0, so overwriting them can have unpredictable consequences
- In some situations, however, dereferencing a null pointer can lead to the execution of arbitrary code

A real-world example

- An exploitable null pointer dereference resulted from a vulnerable version of the libpng library deployed on an ARM-based cell phone
- The libpng library implements its own wrapper to malloc() that returns a null pointer on error or on being passed a 0-byte length argument
- The chunkdata pointer is later used as a destination argument in a call to memcpy()
- Directly writing to a pointer returned from a memory allocation function is more common, but normally less exploitable, than using a pointer as an operand in pointer arithmetic
- If a length field of -1 is supplied, the addition results in 0, and png_malloc() subsequently returns a null pointer, which is assigned to chunkdata
- The subsequent call to memcpy() results in user-defined data overwriting memory starting at address 0
- A write to or read from the memory address 0 will generally reference invalid or unused memory
- In the ARM architecture, the address 0 is mapped in memory and serves as the exception vector table

```
png_charp chunkdata;  
chunkdata = (png_charp)png_malloc(png_ptr, length + 1);
```

Referencing Freed Memory

- It is possible to access freed memory unless all pointers to that memory have been set to NULL or otherwise overwritten
- An example of this programming error is seen in the following loop for a linked list, which dereferences `p` after having first freed the memory referenced by `p`:

```
for (p = head; p != NULL; p = p->next)
    free(p);
```

- The correct way to perform this operation is to save the required pointer before freeing:

```
for (p = head; p != NULL; p = q) {
    q = p->next;
    free(p);
}
```

Reading From Freed Memory

- Reading from freed memory results in undefined behavior – but almost always succeeds without a memory fault because freed memory is recycled by the memory manager
- However, there is no guarantee that the contents of the memory have not been altered
- Although the memory is usually not erased by a call to `free()`, memory managers may use some of the space to manage free or unallocated memory
- If the memory chunk has been reallocated, the entire contents may have been replaced – as a result, these errors may go undetected as the contents of memory may be preserved during testing but modified during operation

Writing to Freed Memory

- Writing to a memory location that has been freed is also unlikely to result in a memory fault – but could result in some serious problems
- If the memory has been reallocated, a programmer may overwrite memory, believing that a memory chunk is dedicated to a particular variable when it is actually being shared
- In this case, the variable contains whatever data was written last
- If the memory has not been reallocated, writing to a free chunk may overwrite and corrupt the data structures used by the memory manager
- This has been used as the basis for an exploit when the data being written is controlled by an attacker

Freeing Memory Multiple Times

- Another dangerous error in managing memory is to free the same memory chunk more than once – the most common scenario being a double-free
- This error is dangerous because it can corrupt the data structures in the memory manager in a manner that is not immediately apparent, but can result in exploitable vulnerabilities

```
x = malloc(n * sizeof(int));  
/* access memory referenced by x */  
free(x);  
y = malloc(n * sizeof(int));  
/* access memory referenced by y */  
free(x);
```

Memory Leaks

- Memory leaks occur when dynamically allocated memory is not freed after it is no longer needed
- Memory leaks can be problematic in long-running processes or exploited in a resource-exhaustion attack (a form of denial-of-service attack)
- If an attacker can identify an external action that causes memory to be allocated but not freed, memory can eventually be exhausted
- Once memory is exhausted, additional allocations fail, and the application is unable to process valid user requests
- This technique might also be used to probe error-recovery code for double-free vulnerabilities and other security flaws

Mitigation Strategies for Heap-based Vulnerabilities

- Memory management flaws leading to heap-based vulnerabilities are problematic as they can have no apparent effect on the correct execution of a program and therefore go undetected
- Several mitigation strategies can be used to eliminate or reduce heap-based vulnerabilities
 - **Set Pointers to NULL:** An obvious technique to reduce vulnerabilities is to set pointers to NULL after the referenced memory is deallocated – reduces vulnerabilities resulting from writing to freed memory and double-free vulnerabilities, but cannot prevent them when multiple pointers reference the same data structure
 - **Consistent Memory Management Conventions:** Use of the same pattern for allocating and freeing memory – for example, allocation and freeing memory in the same module
 - **Randomization:** Randomizing the addresses of blocks of memory returned by the memory manager can make it more difficult to exploit a heap-based vulnerability
 - **Static Analysis Tools:** Diagnose violations of C secure coding rules as a matter of conformance to specifications
 - **Runtime Analysis Tools:** Detect memory violations to eliminate memory-related defects that can lead to heap-based vulnerabilities – these tools typically have high runtime overheads